OBJECT ORIENTED PROGRAMMING

Practical Program File

Course	Object Oriented Programming (CEUE203)
Coverage	Practicals 1 to 5
Contents	Questions followed by submitted Java source code
Source	Original program files supplied by the student

Each program is reproduced from the submitted Java files.

PRACTICAL 1 - Enum, Record and Switch Expression Practice

1. Vending Machine

Question: Write a Java program using an enum Coin with ONE, TWO, FIVE and TEN. Accept coins until the snack price of 15 is paid, then display the change to return.

VendingMachine.java

```
import java.util.Scanner;

public class VendingMachine {
    enum Coins{
ONE,
        TWO,
        FIVE,
        TEN
    }
    public static void main(String[] args) {

        Coins coin1 = Coins.ONE;
        Coins coin2 = Coins.TWO;
        Coins coin5 = Coins.FIVE;
        Coins coin10 = Coins.TEN;

        Scanner sc = new Scanner(System.in);
int total = 0,k;         int snake_price =
15;

        System.out.println(".()");

        while(total < snake_price)
        {
            System.out.println("Enter Coins : ");

            Coins coin = Coins.valueOf(sc.nextLine().toUpperCase());

            switch(coin)
            {
case ONE:
                    total += 1;
```

```
                    System.out.println("You pressed ONE");
break;
                case
TWO:
                    total += 2;
                    System.out.println("You pressed TWO");
break;
                case
FIVE:
                    total += 5;
                    System.out.println("You pressed FIVE");
break;
                case
TEN:
                    total += 10;
                    System.out.println("You pressed TEN");
break;                 default:
                    System.out.println("Invalid Coin. Try Again! ");
            }
        }

        System.out.println("Paid . Change : " + (total - snake_price));

        sc.close();
    }
}
```

```
PS C:\Users\Taksh Patel\Desktop\CODING\SEM--3 coding> cd "c:\Users\Taksh Patel\Desktop\CODING\SEM--3 coding\JAVA_OOP\PR_1\
" ; if ($?) { javac VendingMachine.java } ; if ($?) { java VendingMachine }
.()
Enter Coins :
TEN
You pressed TEN
Enter Coins :
FIVE
You pressed FIVE
Paid . Change : 0
```

2. Toll Booth

Question: Write a Java program using a Vehicle record. Read vehicle details until the user enters done, calculate toll for bike, car and truck using a switch, and display the total toll with the most frequent vehicle type.

TollBooth.java

```
import java.util.Scanner;

public class TollBooth
{
        record Vehicle(String number,String type){}

        public static void main(String[] arg)
        {
                Scanner sc=new Scanner(System.in);

                int totalToll=0;
                int bikeCount=0;
                int carCount=0;
                int truckCount=0;
                System.out.println("If you don't want to continue then type 'done'");
                while(true)
                {
                        System.out.print("Enter vehicle number  : ");
                        String number=sc.nextLine();
                        if(number.equalsIgnoreCase("done"))
                        {
                                break;
```

```
                }
                System.out.print("Enter vehicle type  : ");
                String type=sc.nextLine().toLowerCase();
                Vehicle vehicle=new Vehicle(number,type);
                int toll;

                switch(vehicle.type())
                {
                        case "bike" ->
                        {
                                bikeCount++;
                        toll=20;
                        }
                        case "car" ->
                        {
                                carCount++;
                        toll=50;
                        }
                        case "truck" ->
                        {
                                truckCount++;
                                toll=150;
                        }
                        default ->
                        {
                                System.out.println("Invalid vehicle");
                        toll=0;
                        }
                }
                totalToll+=toll;
        }

        String f;
        if(bikeCount>=carCount&&bikeCount>=truckCount)
        {
                f="bike";
        }
        else if(carCount>=bikeCount&&carCount>=truckCount)
        {
                f="car";
        }
        else
        {
                f="truck";
        }
        System.out.println("\n:::::::::::::: Toll Booth ::::::::::::::");
        System.out.println("Bike Count     : "+bikeCount);
        System.out.println("Car Count      : "+carCount);
        System.out.println("Truck Count    : "+truckCount);
        System.out.println("Total Toll     : "+totalToll);
        System.out.println("Most Frequent  : "+f);

        sc.close();
    }
}
```

```
If you don't want to continue then type 'done'
Enter vehicle number  : 001
Enter vehicle type  : bike
Enter vehicle number  : 002
Enter vehicle type  : car
Enter vehicle number  : done

:::::::::::::: Toll Booth ::::::::::::::
Bike Count     : 1
Car Count      : 1
Truck Count    : 0
Total Toll     : 70
Most Frequent  : bike
```

MiniBank.java import

java.util.Scanner;

```
public class MiniBank {

    record BankInfo(String name, String branch) {
    }

    enum MenuOption {
        OPEN_ACCOUNT,
        DEPOSIT,
        WITHDRAW,
        TRANSFER,
        EXIT
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        BankInfo bank = new BankInfo("MiniBank", "Ahmedabad Branch");
        boolean running = true;

        System.out.println("Welcome to " + bank.name());
        System.out.println("Branch: " + bank.branch());
```

```
        while (running) {
            System.out.println("\n1. Open Account");
            System.out.println("2. Deposit");
            System.out.println("3. Withdraw");
            System.out.println("4. Transfer");
            System.out.println("5. Exit");
            System.out.print("Enter choice: ");

            int choice = scanner.nextInt();

            MenuOption option = switch (choice) {
case 1 -> MenuOption.OPEN_ACCOUNT;
case 2 -> MenuOption.DEPOSIT;                case 3 -
> MenuOption.WITHDRAW;                case 4 ->
MenuOption.TRANSFER;                case 5 ->
MenuOption.EXIT;
                default -> null;
            };

            if (option == null) {
                System.out.println("Invalid choice. Try again.");
                continue;
            }

            switch (option) {
                case OPEN_ACCOUNT -> System.out.println("Open account - to be implemented later.");
case DEPOSIT -> System.out.println("Deposit - to be implemented later.");                case
WITHDRAW -> System.out.println("Withdraw - to be implemented later.");                case
TRANSFER -> System.out.println("Transfer - to be implemented later.");
                case EXIT -> {
                    System.out.println("Thank you for using MiniBank.");
                    running = false;
                }
```

```
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter choice: 4
Transfer - to be implemented later.

1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter choice: 5
Thank you for using MiniBank.
```

```
        }
    }

    scanner.close();
  }
}
```


```
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter choice: 4
Transfer - to be implemented later.

1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter choice: 5
Thank you for using MiniBank.
```

Key Questions and Answers

1. What is the role of the JVM, and what file does the javac compiler produce?

Answer: The Java Virtual Machine runs Java bytecode and provides services such as memory management. The javac compiler converts a .java source file into a .class bytecode file.

2. How does a switch expression differ from a traditional switch statement?

Answer: A switch expression produces a value that can be assigned or returned. Arrow cases do not fall through, while a traditional switch controls statements and often needs break to prevent fall-through.

3. Why are enum and record suitable for the MiniBank menu and BankInfo?

Answer: An enum is suitable for a fixed, named set of menu options. A record is a compact way to hold read-only data such as the bank name and branch.

PRACTICAL 2 - Classes, Constructors, Encapsulation and Static Members

1. Smart Thermostat

Question: Create a thermostat class with a location, temperature range from 16 to 30, constructor chaining, methods to raise and lower temperature, and a static count of active thermostats.

Thermostate.java (original submitted filename)

```
public class Thermostate {
private String location;     private
int temperature;     private static
final int MIN=16;     private static
final int MAX=30;     private static
int activeCount=0;
    Thermostate(String l,int t)
    {
        activeCount++;
this.location=l;         temperature=t;
        if(temperature<=MAX && temperature>=MIN){}
else
            temperature=22;
    }
    Thermostate(String location)
    {
        this(location,22);
    }
    public void raise()
    {
        if(temperature<=MAX)
temperature++;
        else
            System.out.println("maximum temperature");
    }
    public void lower()
    {
        if(temperature>=MIN)
temperature--;
        else
            System.out.println("minimum temperature");
    }
    public int getTemprature()
    {
        return temperature;
    }
    public static int getActiveCounts()
    {
```

```
        return activeCount;
    }
    public static void main(String[] args) {
Thermostate t1 = new Thermostate("HALL",22);
        Thermostate t2 = new Thermostate("KITCHEN");
System.out.println("Temprature Increases  : ");
        for(int i=0;i<=10;i++)
        {
            t1.raise();
            System.out.println(t1.getTemprature());
        }
        System.out.println("Temprature Decreases  : ");
        for(int i=0;i<=20;i++)
        {
            t1.lower();
            System.out.println(t1.getTemprature());
        }
        System.out.println("Total Active counts  : "+t1.getActiveCounts());
} }
```

```
Temprature Increases  :
23
24
25
26
27
28
29
30
31
maximum temperature
31
maximum temperature
31
Temprature Decreases  :
30
29
28
27
26
25
24
23
22
21
20
19
18
17
16
15
minimum temperature
15
minimum temperature
15
minimum temperature
15
minimum temperature
15
minimum temperature
15
Total Active counts  : 2
```

2. Cinema Show

Question: Create a CinemaShow class that books and cancels seats without exceeding its capacity. Use constructors, instance fields and a static total of successful bookings.

CinemaShow.java

```
public class CinemaShow {
     private String title;
private int availableseats;
private final int capacity;
private static int booked=0;

    CinemaShow(String title,int capacity)
    {
        this.title=title;
this.capacity=capacity;
this.availableseats=capacity;
    }
    CinemaShow(String title)
    {
        this(title,100);
    }
    public String book(int n)
    {
        if (n<=availableseats)
        {
            availableseats-= n;
booked+= n;             return
"Booked";
        }
else          {
            return "Not Available";
        }
}
    public void cancel(int n)
    {
        availableseats+= n;
if (availableseats>capacity)
        {
            availableseats=capacity;
        }
    }
    public int getSeatsAvailable()
    {
        return availableseats;
    }
    public static int getbooked()
    {
        return booked;
    }
    public static void main(String[] args) {

        CinemaShow show = new CinemaShow("Avengers", 100);

        System.out.println("Movie: "+show.title);
        System.out.println("Initial Seats: "+show.getSeatsAvailable());

        System.out.println("Book 30 Seats : "+show.book(30));
        System.out.println("Seats Available : "+show.getSeatsAvailable());

        System.out.println("Book 50 Seats : "+show.book(50));
        System.out.println("Seats Available : "+show.getSeatsAvailable());

        System.out.println("Book 40 Seats : "+show.book(40));
        System.out.println("Seats Available : "+show.getSeatsAvailable());
```

```
        show.cancel(20);
        System.out.println("Cancel 20 Seats");
        System.out.println("Seats Available : " + show.getSeatsAvailable());

        show.cancel(50);
        System.out.println("Cancel 50 Seats");
        System.out.println("Seats Available : " + show.getSeatsAvailable());
        System.out.println("Total booked seats : " + CinemaShow.getbooked());
    } }
```

```
Movie: Avengers
Initial Seats: 100
Book 30 Seats : Booked
Seats Available : 70
Book 50 Seats : Booked
Seats Available : 20
Book 40 Seats : Not Available
Seats Available : 20
Cancel 20 Seats
Seats Available : 40
Cancel 50 Seats
Seats Available : 90
Total booked seats : 80
```

MiniBank.java public

class MiniBank {

public static void main(String[] args) {

Customer customer1 = new Customer("Riya", "riya@gmail.com", "9876543210");

Customer customer2 = new Customer("Dev", "dev@gmail.com", "9876543211");

Customer customer3 = new Customer("Aman", "aman@gmail.com", "9876543212");

Account[] accounts = { new Account(customer1.getName(), 5000), new Account(customer2.getName(), 3000), new Account(customer3.getName())

};

accounts[0].deposit(2000); accounts[0].withdraw(3000);

accounts[1].deposit(1000); accounts[1].withdraw(5000);

```
        accounts[2].deposit(500);

        System.out.println("Account balances:");
        for (Account account : accounts) {
            System.out.println(account.getAccountNumber() + " - "
                    + account.getOwnerName() + " - Rs. " + account.getBalance());
        }
    }
}
```

Account.java public

```
class Account {

    private static int accountCounter = 1;

    private final String accountNumber;
private String ownerName;     private
long balance;     private boolean
active;

    public Account(String ownerName, long openingBalance) {
this.accountNumber = generateAccountNumber();
this.ownerName = ownerName;         this.balance =
openingBalance;
        this.active = true;
    }

    public Account(String ownerName) {
this(ownerName, 0);
    }

    private static String generateAccountNumber() {
 return String.format("AC%04d", accountCounter++);
```

```
  }

  public void deposit(long amount) {
      if (amount > 0) {
balance = balance + amount;
      }
  }

   public boolean withdraw(long amount) {
if (amount > 0 && amount <= balance) {
balance = balance - amount;
          return true;
      }
      return false;
  }

   public String getAccountNumber() {
return accountNumber;
  }

   public String getOwnerName() {
return ownerName;
  }

   public long getBalance() {
return balance;
  }

   public boolean isActive() {
return active;
  }
}
```

Customer.java public
class Customer {

private static long customerCounter = 101;

private final String customerId;
private String name; private
String email; private String
mobile;

public Customer(String name, String email, String mobile) {
this.customerId = generateCustomerId(); this.name =
name; this.email = email; this.mobile = mobile;
}

private static String generateCustomerId() {
return "CUST" + customerCounter++;
}

public String getCustomerId() {
return customerId;
}

public String getName() {
return name;
}

public String getEmail() {
return email;
}
public String getMobile() {
return mobile;
}

```
}
```

```
PS C:\Users\Taksh Patel> cd "c:\Users\Taksh Patel\Desktop\CODING\SEM--3 coding\JAVA_OOP\PROJECT\Practical_
Account balances:
AC0001 - Riya - Rs. 4000
AC0002 - Dev - Rs. 4000
AC0003 - Aman - Rs. 500
```

Key Questions and Answers

1. What is encapsulation, and how do private fields with public getters enforce it?

Answer: Encapsulation keeps data and its operations together. Private fields cannot be changed directly outside the class; getters expose values safely, while methods such as deposit and withdraw control valid changes.

2. Why is accountNumber declared final?

Answer: An account number identifies one account. Making it final prevents it from being reassigned after the constructor has created the account.

3. How do static and instance fields differ when generating ids?

Answer: A static counter belongs to the class and is shared by all objects, so it can generate unique ids. An instance field belongs to one object and cannot maintain a shared sequence.

PRACTICAL 3 - Object Methods, Equality, Nested Class and Clone

---

1. Distinct Points

Question: Create a Point class with x and y coordinates. Override toString(), equals() and hashCode(), then use a Driver class to examine repeated points in an array.

Point.java

```
import java.util.Objects; public
class Point {         private
int x,y;

    public Point(int x,int y)
    {
this.x=x;
this.y=y;
    }

    @Override
    public String toString()
    {
        return("x : "+x+"y : "+y);
    }

    @Override
    public boolean equals(Object obj) {
if (this==obj)             return true;

        if (!(obj instanceof Point))
return false;

        Point p =(Point)obj;

        return x==p.x && y == p.y;
    }

    @Override
    public int hashCode()
    {
        return Objects.hash(x, y);
    }
```

```
}
```

Driver.java (Distinct Points)

```
public class Driver
{
	public static void main(String[] args)
	{
		Point[] points={
			new Point(2,3),
			new Point(4,5),
			new Point(2,3),
			new Point(6,7),
			new Point(4,5)
		};
		int distinct=0;
		for(int i=0;i<points.length;i++)
		{
			boolean found=false;
		for(int j=0;j<i;j++)
			{
				if(points[i].equals(points[j]))
				{
					found=true;
				break;
				}
			}
			if(found)
			distinct++;
		}

		System.out.println("Distinct: "+distinct);
	}
}
```

```
Distinct: 2
```

2. Duplicate Card Check

Question: Create a Card class with rank and suit. Override toString(), equals() and hashCode(), then check a Card array and print the first duplicate card found.

Card.java

```
import java.util.Objects;

public class Card
{
	private String rank;
	private String suit;

	public Card(String rank,String suit)
	{
		this.rank=rank;
	this.suit=suit;
	}
	@Override
	public String toString()
	{
		return rank+" of "+suit;
	}
	@Override
	public boolean equals(Object obj)
	{
		if(this==obj)
			return true;
```

```
                if(!(obj instanceof Card))
                        return false;

                Card c=(Card)obj;
                return rank.equals(c.rank) && suit.equals(c.suit);
        }
        @Override
        public int hashCode()
        {
                return Objects.hash(rank,suit);
        }
}
```

Driver1.java (Duplicate Card Check)

```
public class Driver1 {
        public static void main(String[] args) {

        Card[] cards = new Card[5];
         cards[0] = new Card("Ace", "Spades");
cards[1] = new Card("King", "Hearts");
cards[2] = new Card("Queen", "Diamonds");
cards[3] = new Card("Ace", "Spades");
cards[4] = new Card("Jack", "Clubs");

        boolean flag = false;
        for (int i = 0; i < cards.length; i++)
        {
            for (int j = 0; j < i; j++)
            {
                if (cards[i].equals(cards[j]))
                {
                    System.out.println("Duplicate found  : " + cards[i]);
flag = true;                    break;
                }
}             if
(flag==true)
break;         }
if(flag==false)
            System.out.println("There is no Any Duplicate");
    }
}
```

```
Duplicate found  : Ace of Spades
```

Part B Project Work MiniBank

MiniBank Project: This Practical 3 project extends MiniBank with object methods, equality, a nested Address class and cloning. The project source below includes the main program, Account and Customer classes.

MiniBank.java

```
public class MiniBank {

    public static void main(String[] args) {

        Account a1 = new Account("Riya", 4000);
        Account a2 = new Account(a1);

        Customer customer = new Customer(
            "Riya",
            "riya@gmail.com",
            "9876543210"
        );

        Customer.Address address = new Customer.Address(
            "12 Main Road",
            "Delhi",
            "110001"
        );

        customer.setAddress(address);

        System.out.println("Account 1: " + a1);
        System.out.println("Account 2: " + a2);
        System.out.println("a1.equals(a2): " + a1.equals(a2));

        Object obj = a1;
        System.out.println("obj instanceof Account: " + (obj instanceof Account));

        Customer copy = customer.clone();
        System.out.println("Customer clone: " + copy.getCustomerId());
        System.out.println("Address: " + customer.getAddress());
    }
}
```

Account.java

```
import java.util.Objects;

public class Account {

    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static long accountCounter = 0;

    private static String generateAccountNumber() {
        accountCounter++;
        return String.format("AC%04d", accountCounter);
    }
```

```
public Account(String ownerName, long openingBalance) {
    this.accountNumber = generateAccountNumber();
    this.ownerName = ownerName;
    this.balance = openingBalance;
    this.active = true;
}

public Account(String ownerName) {
    this(ownerName, 0);
}

// Used only to create a second object with the same account number for equality testing.
public Account(Account other) {
    this.accountNumber = other.accountNumber;
    this.ownerName = other.ownerName;
    this.balance = other.balance;
    this.active = other.active;
}

public void deposit(long amount) {
    if (amount > 0) {
        balance += amount;
    }
}

public boolean withdraw(long amount) {
    if (amount > 0 && balance >= amount) {
        balance -= amount;
        return true;
    }
    return false;
}

public String getAccountNumber() {
    return accountNumber;
}

public String getOwnerName() {
    return ownerName;
}

public long getBalance() {
    return balance;
}

public boolean isActive() {
    return active;
}

@Override
public String toString() {
    return accountNumber + " | " + ownerName + " | " + balance;
}

@Override
public boolean equals(Object o) {
    if (this == o) {
```

```
            return true;
        }

        if (!(o instanceof Account)) {
            return false;
        }

        Account other = (Account) o;
        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

Customer.java

```
public class Customer implements Cloneable {

    private String name;
    private String email;
    private String mobile;
    private final String customerId;

    private Address address;

    private static long customerCounter = 100;

    private static String generateCustomerId() {
        customerCounter++;
        return "CUST" + customerCounter;
    }

    public Customer(String name, String email, String mobile) {
        this.name = name;
        this.email = email;
        this.mobile = mobile;
        this.customerId = generateCustomerId();
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }

    public String getMobile() {
        return mobile;
    }

    public String getCustomerId() {
        return customerId;
    }

    public Address getAddress() {
```

```
        return address;
    }

    public void setAddress(Address address) {
        this.address = address;
    }

    public static class Address {

        private String line;
        private String city;
        private String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
            this.city = city;
            this.pincode = pincode;
        }

        public String getLine() {
            return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
            return pincode;
        }

        @Override
        public String toString() {
            return line + ", " + city + " - " + pincode;
        }
    }

    @Override
    public Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);
        }
    }
}
```

```
Account 1: AC0001 | Riya | 4000
Account 2: AC0001 | Riya | 4000
a1.equals(a2): true
obj instanceof Account: true
Customer clone: CUST101
Address: 12 Main Road, Delhi - 110001
```

Key Questions and Answers

1. Why must equals() and hashCode() be overridden together?

Answer: Equal objects must return the same hash code. Hash-based collections such as HashSet and HashMap use the hash code first and equals() to confirm a match.

2. What does the default Object.toString() return, and why override it?

Answer: The default text usually contains the class name and a hash value. Overriding toString() gives a readable description of the object, which is more useful for printing and debugging.

3. What is the difference between a static nested class and an inner class?

Answer: A static nested class has no link to an outer object. A non-static inner class belongs to an outer object and can directly access its instance members.

PRACTICAL 4 - Strings and Regular Expressions

1. Password Strength Checker

Question: Write a PasswordChecker program that checks password length, uppercase letters, digits and special characters. Count the passed rules and classify the password as Weak, Medium or Strong.

PasswordChecker.java

```
public class PasswordChecker {

    public static int checkRules(String pw) {
int count=0;

        if(pw.length()>=8) {
            System.out.println("Length >= 8       : Passed");
count++;
        } else
            System.out.println("Length >= 8       : Failed");

        if(pw.matches(".*[A-Z].*")) {
            System.out.println("Uppercase         : Passed");
count++;
        } else
            System.out.println("Uppercase         : Failed");

        if(pw.matches(".*[0-9].*")) {
            System.out.println("Digit             : Passed");
count++;
        } else
            System.out.println("Digit             : Failed");

        if(pw.matches(".*[^a-zA-Z0-9].*")) {
            System.out.println("Special character : Passed");
count++;
        } else
            System.out.println("Special character : Failed");

        return count;
    }
    public static String strength(String pw) {
int count=checkRules(pw);
         if(count<=1)
return "Weak";         else
if(count<=3)
return "Medium";
else
            return "Strong";
    }
}
```

Driver.java (Password Strength Checker)

```
public class Driver {
    public static void main(String[] args) {
        String[] passwords={"abc","Abcd1234!"};

        for(String pw:passwords) {
            System.out.println("\nPassword: "+pw);
            System.out.println("Strength: "+PasswordChecker.strength(pw));
        }
} }
```

```
Password: abc
Length >= 8       : Failed
Uppercase         : Failed
Digit             : Failed
Special character : Failed
Strength: Weak

Password: Abcd1234!
Length >= 8       : Passed
Uppercase         : Passed
Digit             : Passed
Special character : Passed
Strength: Strong
```

2. Chat-log filter

Chatfilter.java

```
public class Chatfilter {

    public static String[] splitLine(String line) {

        String[] parts=line.split(" ", 3);

        if (parts.length<3) {
return null;
        }

        return parts;
    }
}
```

Driver1.java import

```
java.util.Scanner;

public class Driver1
{
    public static void main(String[] args)
    {
        Scanner sc=new Scanner(System.in);    String[] log={
```

```
            "10:05 alice Hello there",
            "10:06 bob How are you?",
             "10:07"
        };

        System.out.print("Enter keyword: ");
        String keyword=sc.nextLine();

        int matches=0;
        StringBuilder report=new StringBuilder();

        for(String line:log)
        {
            String[] parts=Chatfilter.splitLine(line);

            if(parts==null)
            {
                continue;
            }

            if(parts[2].toLowerCase().contains(keyword.toLowerCase()))
            {
                matches++;

                report.append(parts[0])
.append(" ")
                      .append(parts[1])
                     .append(": ")
                     .append(parts[2])
                     .append("\n");
            }
        }
```

```
        System.out.println("Matches: "+matches);
        System.out.println(report);

        sc.close();
    }
}
```

```
Enter keyword: Hello
Matches: 1
10:05 alice: Hello there
```

MiniBank.java public

```
class MiniBank {

    public static void main(String[] args) {

        System.out.println("Mobile valid: " +
            Validator.isValidMobile("9876543210"));

        System.out.println("Mobile valid: " +
            Validator.isValidMobile("12345"));

        System.out.println("Email valid: " +
            Validator.isValidEmail("riya@gmail.com"));

        System.out.println("PAN valid: " +
            Validator.isValidPan("ABCDE1234F"));

        System.out.println("IFSC valid: " +
            Validator.isValidIfsc("SBIN0001234"));

        Command command =
            CommandParser.parse("DEPOSIT AC0001 500");
```

```
        System.out.println("Type: " + command.type());
        System.out.println("Account: " + command.accountNumber());
        System.out.println("Amount: " + command.amount());

        Account account = new Account("Riya", 5000);

        System.out.println(
            StatementFormatter.buildStatement(account)
        );
    }
}
```

Account.java

```
import java.util.Objects;

public class Account {

    private final String accountNumber;
private String ownerName;     private
long balance;     private boolean
active;

    private static long accountCounter = 0;

    private static String generateAccountNumber() {
accountCounter++;        return
String.format("AC%04d", accountCounter);
    }

    public Account(String ownerName, long openingBalance) { this.accountNumber =
    generateAccountNumber();    this.ownerName = ownerName;     this.balance = openingBalance;
        this.active = true;
    }
```

```
    public Account(String ownerName) {
this(ownerName, 0);
    }

    public Account(Account other) {
this.accountNumber = other.accountNumber;
this.ownerName = other.ownerName;
        this.balance = other.balance;
this.active = other.active;
    }

    public void deposit(long amount) {
        if (amount > 0) {
balance += amount;
        }
    }

    public boolean withdraw(long amount) {
if (amount > 0 && balance >= amount) {
balance -= amount;
            return true;
        }
       return false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
return ownerName;
    }
```

```
    public long getBalance() {
return balance;
    }

    public boolean isActive() {
return active;
    }

    @Override
    public String toString() {        return accountNumber + " | " +
ownerName + " | " + balance;
    }

    @Override     public boolean
equals(Object o) {
        if (this == o) {
return true;
        }

        if (!(o instanceof Account)) {
return false;
        }

        Account other = (Account) o;   return
    accountNumber.equals(other.accountNumber);
    }
    @Override
    public int hashCode() {     return
    Objects.hash(accountNumber);
    }
}
```

Customer.java public class Customer
implements Cloneable {

private String name; private
String email; private String
mobile; private final String
customerId;

private Address address;

private static long customerCounter = 100;

private static String generateCustomerId() {
customerCounter++; return "CUST" +
customerCounter;
}

public Customer(String name, String email, String mobile) {
this.name = name; this.email = email; this.mobile =
mobile; this.customerId = generateCustomerId();
}

public String getName() {
return name;
}
public String getEmail() {
return email;
}

public String getMobile() {
return mobile;
}

```
    public String getCustomerId() {
return customerId;
    }

    public Address getAddress() {
return address;
    }

    public void setAddress(Address address) {
this.address = address;
    }

    public static class Address {

        private String line;
private String city;         private
String pincode;

        public Address(String line, String city, String pincode) {
            this.line = line;
 this.city = city;
             this.pincode = pincode;
       }
       public String getLine() {
           return line;
        }

        public String getCity() {
            return city;
        }

        public String getPincode() {
return pincode;
```

```
        }

        @Override        public String toString() {
return line + ", " + city + " - " + pincode;
        }
    }

    @Override     public
Customer clone() {
        try {
            return (Customer) super.clone();        }
catch (CloneNotSupportedException e) {
throw new AssertionError(e);
        }
    }
}
```

Command.java public

```
record Command(
TransactionType type,
String accountNumber,
long amount
) {}
```

CommandPraser.java public

```
class CommandParser {

    public static Command parse(String line) {

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3) {
            throw new IllegalArgumentException(
```

```
            "Command must have: TYPE ACCOUNT_NUMBER AMOUNT"
        );
    }

    TransactionType type =
        TransactionType.valueOf(parts[0].toUpperCase());

    String accountNumber = parts[1];
long amount = Long.parseLong(parts[2]);

    return new Command(type, accountNumber, amount);
  }
}
```

StatementFormatter.java public
class StatementFormatter {

```
  public static String buildStatement(Account account) {

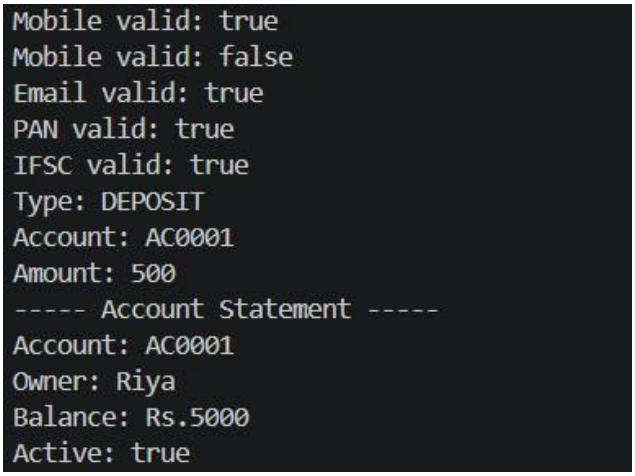
 StringBuilder statement = new StringBuilder();

    statement.append("----- Account Statement -----\n");      statement.append("Account: ")
            .append(account.getAccountNumber())
            .append("\n");
    statement.append("Owner: ")
            .append(account.getOwnerName())
            .append("\n");
    statement.append("Balance: Rs.")
            .append(account.getBalance())
            .append("\n");
    statement.append("Active: ")
            .append(account.isActive())
            .append("\n");
```

```
        return statement.toString();
    }
}
```

TransactionType.java public

```
enum TransactionType {
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```



Key Questions and Answers

1. What is the difference between String, StringBuilder and StringBuffer?

Answer: String is immutable. StringBuilder is mutable and efficient for repeated changes in single-threaded programs. StringBuffer is mutable and synchronized for shared use across threads.

2. What does the regular expression [6-9][0-9]{9} match, and why use anchors?

Answer: It matches a ten-digit mobile number beginning with 6, 7, 8 or 9. The anchors ^ and $ ensure that the complete input matches, with no extra characters.

3. Why should input be validated before it is used?

Answer: Early validation prevents invalid data from reaching later logic. It keeps objects in a valid state and makes errors clear to the user at the point of entry.

Practical-5

1. Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle, Triangle. Put a mix in a Shape[] and loop once to print each area and the running total, tracking the largest — one loop handling every shape through polymorphism.

Circle.java

```
public class Circle extends Shape {

    private double radius;

    public Circle(double radius) {
this.radius = radius;
    }

    @Override     public double area()
{        return Math.PI * radius *
radius;
    }
}
```

Rectangle.java

```
public class Rectangle extends Shape {

    private double length;
private double width;

    public Rectangle(double length, double width) {
this.length = length;         this.width = width;
    }
```

```
    @Override     public
double area() {
return length * width;
    }
}
```

Shape.java

```
public abstract
class Shape {

    public abstract double area();
}
```

Triangle.java

```
public class Triangle
extends Shape {

    private double base;
private double height;

    public Triangle(double base, double height) {
this.base = base;         this.height = height;
    }

    @Override     public double
area() {         return 0.5 * base
* height;
    }
}
```

Main.java

```
public
class Main {
public Main() {
```

```
  }

  public static void main(String[] var0) {
    Shape[] var1 = new Shape[]{new Circle((double)5.0F), new Rectangle((double)4.0F,
(double)6.0F), new Triangle((double)8.0F, (double)3.0F)};
double var2 = (double)0.0F;       double var4 =
(double)0.0F;

    for(Shape var9 : var1) {
double var10 = var9.area();
      System.out.println("Area = " + var10);
var2 += var10;          if (var10 > var4) {
var4 = var10;
      }
    }

    System.out.println("Total Area = " + var2);
    System.out.println("Largest Area = " + var4);
  }
}
```

```
Area = 78.53981633974483
Area = 24.0
Area = 12.0
Total Area = 114.53981633974483
Largest Area = 78.53981633974483
```

2. Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.

Employee.java public abstract
class Employee {

    protected String name;
    protected int id;

    public Employee(String name, int id) {
this.name = name;
        this.id = id;
    }

    public abstract double monthlySalary();
}
FullTime.java public class FullTime
extends Employee {

    private double salary;

    public FullTime(String name, int id, double salary) {
super(name, id);   this.salary = salary;
    }

    @Override   public double
monthlySalary() {
        return salary;
    }
}
Intern.java public class Intern extends
Employee {   private double stipend;

    public Intern(String name, int id, double stipend) {
super(name, id);   this.stipend = stipend;
    }

```
    @Override     public double
monthlySalary() {
        return stipend;
    }
}
PartTime.java public class PartTime
extends Employee {

    private double hours;
private double rate;

    public PartTime(String name, int id, double hours, double rate) {
super(name, id);         this.hours = hours;         this.rate = rate;
    }

    @Override     public double
monthlySalary() {
        return hours * rate;
    }
}
Main.java public
class Main { public
static void
main(String[] args)
{

        Employee[] employees = {         new
FullTime("Rahul", 101, 50000),             new
PartTime("Amit", 102, 80, 300),             new
Intern("Priya", 103, 15000)
        };
```

```
        double total = 0;

        for (Employee e : employees) {

            double salary = e.monthlySalary();

            System.out.println(
                e.name + " (ID: " + e.id + ") Salary = Rs. " + salary
            );

            total += salary;

            if (e instanceof Intern) {
                System.out.println("Note: This employee is an Intern.");
            }
        }

        System.out.println("------------------------");
        System.out.println("Total Payroll = Rs. " + total);
    }
}
```

```
Rahul (ID: 101) Salary = Rs. 50000.0
Amit (ID: 102) Salary = Rs. 24000.0
Priya (ID: 103) Salary = Rs. 15000.0
Note: This employee is an Intern.
------------------------
Total Payroll = Rs. 89000.0
```

Project Work

Prat : B

MiniBank.java public

class MiniBank {

```
    public static void main(String[] args) {
```

```
        Account[] accounts = {              new
SavingsAccount("Riya", 10000, 2000),              new
CurrentAccount("Aman", 5000, 3000),              new
FixedDepositAccount("Neha", 20000)
        };

        double totalInterestRate = 0;

        for (Account account : accounts) {

            System.out.println(
account.getAccountNumber() +
                " | " +
                account.getOwnerName() +
                " | Interest Rate: " +
account.interestRate()
            );

            totalInterestRate += account.interestRate();

            if (account instanceof FixedDepositAccount) {
                System.out.println("Note: Fixed deposit is locked.");
            }
        }

        System.out.println("Total of rates: " + totalInterestRate);

        System.out.println(
"Savings withdrawal: " +
accounts[0].withdraw(7000)
        );
```

```
        System.out.println(
"Current withdrawal: " +
accounts[1].withdraw(7000)
        );

        System.out.println(
            "Fixed deposit withdrawal: " +
accounts[2].withdraw(1000)
        );
    }
}
```

Account.java

```
import java.util.Objects;

public abstract class Account {

   private final String accountNumber;
private String ownerName; private
long balance;
    private boolean active;

    private static long accountCounter = 0;

    private static String generateAccountNumber() {
accountCounter++;         return
String.format("AC%04d", accountCounter);
    }

    public Account(String ownerName, long openingBalance) {
this.accountNumber = generateAccountNumber();
```

```
this.ownerName = ownerName;        this.balance =
openingBalance;
        this.active = true;
    }

    public Account(String ownerName) {
this(ownerName, 0);
    }

    public abstract double interestRate();

    public abstract boolean canWithdraw(long amount);

    public void deposit(long amount) {
        if (amount > 0) {
balance += amount;
        }
    }

   public boolean withdraw(long amount) {
   if (canWithdraw(amount)) {     balance -
   = amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber() {
return accountNumber;
    }

    public String getOwnerName() {
return ownerName;
```

```
    }

    public long getBalance() {
return balance;
    }

    public boolean isActive() {
return active;
    }

    @Override
    public String toString() {        return accountNumber + " | " +
ownerName + " | " + balance;
    }

    @Override     public boolean
equals(Object o) {
        if (this == o) {
    return true;
       }

        if (!(o instanceof Account)) {
            return false;
        }

        Account other = (Account) o;        return
accountNumber.equals(other.accountNumber);
    }

    @Override     public int hashCode() {
return Objects.hash(accountNumber);
    }
}
```

```
}
Customer.java public class Customer
implements Cloneable {

    private String name;     private
String email;     private String
mobile;     private final String
customerId;

    private Address address;

    private static long customerCounter = 100;

    private static String generateCustomerId() {
customerCounter++;         return "CUST" +
customerCounter;
    }

    public Customer(String name, String email, String mobile) {
        this.name = name;
        this.email = email; this.mobile = mobile;
        this.customerId =
        generateCustomerId();
    }

    public String getName() {
return name;
    }

    public String getEmail() {
return email;
    }
```

```
    public String getMobile() {
return mobile;
    }

    public String getCustomerId() {
return customerId;
    }

    public Address getAddress() {
return address;
    }

    public void setAddress(Address address) {
this.address = address;
    }

    public static class Address {

        private String line;
        private String city; private
        String pincode;


       public Address(String line, String city, String pincode) {
            this.line = line;
this.city = city;
            this.pincode = pincode;
        }

        public String getLine() {
            return line;
        }
```

```
        public String getCity() {
            return city;
        }

        public String getPincode() {
return pincode;
        }

        @Override         public String toString() {
return line + ", " + city + " - " + pincode;
        }
    }

    @Override     public
Customer clone() {
        try {
            return (Customer) super.clone();
        } catch (CloneNotSupportedException e) {
     throw new AssertionError(e);
        }
    }
}
```

Command.java public

```
record Command(
    TransactionType type,
String accountNumber,     long
amount
) {}
```

CommandPraser.java public

```
class CommandParser {
```

```
    public static Command parse(String line) {

        String[] parts = line.trim().split("\\s+");

        if (parts.length != 3) {
            throw new IllegalArgumentException(
                "Command must have: TYPE ACCOUNT_NUMBER AMOUNT"
            );
        }

        TransactionType type =
            TransactionType.valueOf(parts[0].toUpperCase());

        String accountNumber = parts[1];
long amount = Long.parseLong(parts[2]);

     return new Command(type, accountNumber, amount);
 }
}
```

StatementFormatter.java

```
public
class StatementFormatter {

    public static String buildStatement(Account account) {

        StringBuilder statement = new StringBuilder();

        statement.append("----- Account Statement -----\n");
statement.append("Account: ")
                .append(account.getAccountNumber())
                .append("\n");
        statement.append("Owner: ")
                .append(account.getOwnerName())
```

```
                .append("\n");
        statement.append("Balance: Rs.")
                .append(account.getBalance())
                .append("\n");
        statement.append("Active: ")
                .append(account.isActive())
                .append("\n");

        return statement.toString();
    }
}
```

TransactionType.java public

```
enum TransactionType {
    DEPOSIT,
    WITHDRAW,
    TRANSFER
}
```

CurrentAccount.java public class

```
CurrentAccount extends Account {

    private long overdraftLimit;

    public CurrentAccount(
String ownerName,        long
openingBalance,        long
overdraftLimit
    ) {
        super(ownerName, openingBalance);
        this.overdraftLimit = overdraftLimit;
    }
```

```
    @Override     public double
interestRate() {
        return 0.0;
    }

    @Override     public boolean canWithdraw(long
amount) {         return amount > 0 &&
getBalance() - amount >= -overdraftLimit;
    }

    public long getOverdraftLimit() {
return overdraftLimit;
    }
}

FixedDepositAccount.java public class
FixedDepositAccount extends Account {     public
FixedDepositAccount(         String ownerName,
long openingBalance
    ) {
        super(ownerName, openingBalance);
    }

    @Override     public double
interestRate() {
        return 7.0;
    }

    @Override     public boolean
canWithdraw(long amount) {
        return false;
    }
}
```

SavingAccount.java public class SavingsAccount extends Account {

```
    private long minBalance;

    public SavingsAccount(
String ownerName,         long
openingBalance,         long
minBalance
    ) {
        super(ownerName, openingBalance);
this.minBalance = minBalance;
    }
```

```
AC0001 | Riya | Interest Rate: 4.0
AC0002 | Aman | Interest Rate: 0.0
AC0003 | Neha | Interest Rate: 7.0
Note: Fixed deposit is locked.
Total of rates: 11.0
Savings withdrawal: true
Current withdrawal: true
Fixed deposit withdrawal: false
```

```
   @Override
    public double interestRate() {
        return 4.0;
    }

    @Override     public boolean
canWithdraw(long amount) {         return amount
> 0 &&                 getBalance() - amount >=
minBalance;
    }
```

```
    public long getMinBalance() {
return minBalance;
    }
}
```

Key Questions and Answers

1. What is the difference between an abstract class and a concrete class?

Answer: An abstract class can declare methods without implementations and cannot be instantiated directly. A concrete class provides the required implementations and can create objects.

2. What is dynamic method dispatch, also called run-time polymorphism?

Answer: When an overridden method is called through a parent-class reference, Java selects the method implementation using the actual object type at run time.

3. What does super do in a subclass constructor?

Answer: super(arguments) calls the parent-class constructor so that the inherited part of the object is initialized first. It must be the first statement in a subclass constructor.